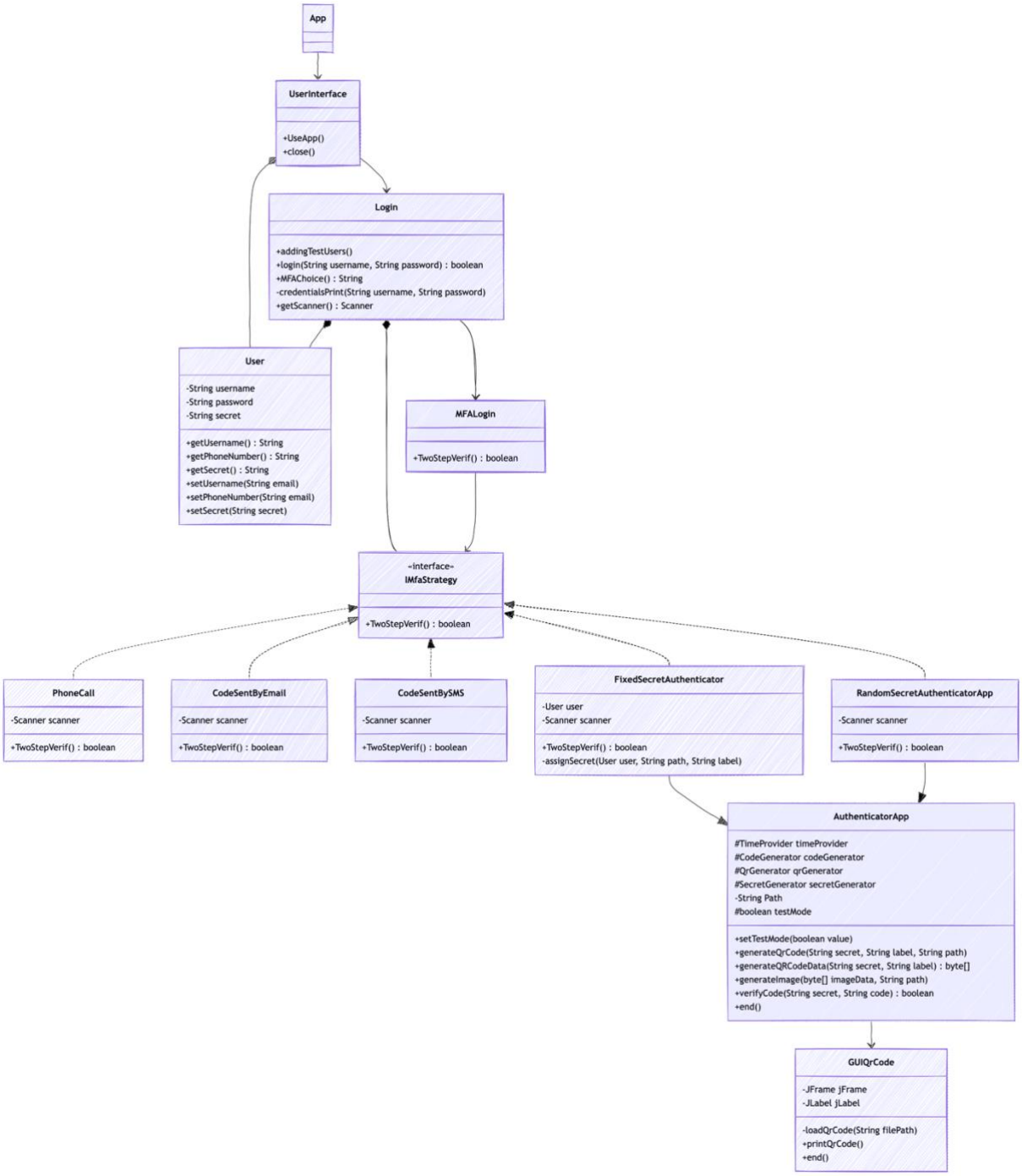
001 - Intermediate Coursework report

1) Design



For the design I will explain the UML diagram, after that I will talk about the design pattern used and finally I will talk about the application of the SOLID Principle in this design.

The UserInterface and the Login class are in a composition relationship with the User class. This means that the classes maintain a list of users.

The Login class is in a composition relationship with the MFALogin class. This is a composition because the Login class instantiate the MFALogin class and owns an instance of this class.
The class RandomSecretAuthentificatorApp and the FixedSecretAuthentificator inherit from AuthentificatorApp. This inheritance is useful to avoid code duplication and to improve maintainability because those two classes need to generate and print QR Code.

Several classes such as PhoneCall, CodeSentBySMS, CodeSentByEmail, RandomSecretAuthentificatorApp, and FixedSecretAuthentificator implement the interface IMFAStrategy. This is useful to use diverse method of multi-factor authentication without modifying the code.

I used the strategy pattern to implement the multi factor authentication. The Strategy pattern nvolves several components to take certain task like the context, the client, the abstract strategy and the concrete strategy.

For the client I chose the MFALogin class. This class can only use the TwoStepVerif method, which is common to all the strategies given by the context, which is the Login class.

The interface IMFAStrategy represents the abstract strategy and only define the TwoStepVerif method. This method will be implemented in the concrete strategy.
For the concrete strategy there are 5 classes: PhoneCall, CodeSentByEmail, CodeSentBySMS, RandomSecretAuthentificatorApp, FixedSecretAuthetificator.

Each of these class implements the TwoStepVerif method and when creating a strategy, a concrete strategy must be instantiated with the abstract strategy as the type.
Thanks to this pattern the only classes that know the exact implementation of the MFA method are the MFA method and if we need to add another MFA method we just need to create a new class that implements IMFAStrategy.

Another benefit of this pattern is that one can instantiate several strategies at runtime with a minimal number of objects.

The SOLID Principle are design principles used to improve the quality of code as well as its maintainability and extensions. This acronym stands for: Single-Responsibility Principle (SRP), Open-Closed Principle (OCP), Liskov Substitution Principle (LSP), Interface Segregation Principle (ISP) and Dependency Inversion Principle (DIP).

The SRP is respected because each class does only one thing, the Login class handles the login without handling the MFA, the MFALogin handle only verification with the MFA method and the class AuthentificatorApp only handle the QR Code generation and verification.

The OCP is respected because to add another method of double authentication one just needs to create a class that implement the interface IMFAStrategy. They don't need to modify any other class.

The LSP is respected because any class that implement IMFAStrategy can be replaced by another class that implement this interface.

The ISP is respected because in the IMFAStrategy there is only one method and it used in all the concrete strategy. So, the interface doesn't force the other class to implement useless method.
The DIP is respected because the class using the strategy depends on an abstract strategy and not on any concrete one.

2) Implementation

In the UserInterface I use two variables testMode and input, the testMode variable is useful to avoid creating a GUI during the test because jacoco cannot properly handle GUI window during test. And for the input which is an inputStream is used to create a single scanner for the whole instance, this is important because if I give the input Stream to other classes and create new scanners out of it the inputStream will be flushed [1]. I also created a close method to close any scanner that is not linked to the system.

The Login class contains 3 variables: testMode, users and scanner. The testMode and scanner variable are provided by the UserInterface and the users variable is a map containing the username as the key and the user associated with this username as the value. I chose a map because the lookup is fast and efficient, but I can't have two users with the same username. The Login class can obtain a map from the UserInterface but add 3 test users in all cases. I chose to first ask for all the credentials before printing if they are correct. As for the other input like the choice of MFA, I put some message in case of invalid value. If the user has the right credentials, they have 3 attempts to connect using MFA.
In the Login class, I instantiate an MFALogin class with only a strategy. The MFALogin class contains a single method that calls the common method of the strategy if the strategy is defined.
For the PhoneCall, CodeSentByEmail, CodeSentBySMS classes I just pass a scanner and ask for information from the user, finally I validate the MFA attempt. There are just some prototypes.
Now I will talk about the AuthentificatorApp class. This class use a Time-based One Time Password (MFA) Library for Java developed by Sam Stevens, Olivier Jaquemat, Paulo Costa and Ralph Schaer [3]. In this class I don't generate secret, but I used some provided as arguments to generate QR Code data and then create an image with the QR Code that I store where I want. I can also verify the connection using the typed code and a secret. In this class I also open a window with the QR Code to help the user scan it, so I needed to create a method to close the window if created. All the methods in this class throw exceptions without catching them because this class needs to be used in another class.
I opened the window with the GUIQrCode class, to do that I used the swing library [4]. In case of missing file path or missing file I created a custom window to tell the user about the situation.
The RandomSecretAthentificatorApp class extends the AuthentificatorApp and implements the IMFAStrategy. So, I have a constructor with all the variable needed for the TOPT [3]. In this class I generate a secret and then I show a QR Code and ask for the code. If the code is correct, I accept the connection. The QR Code is regenerated each time a user tries this method.

The FixedSecretAthentificator class extends the AuthentificatorApp and implements the IMFAStrategy. So, I have the same variables in the constructor as the AuthentificatorApp plus a scanner, a user and the testMode. I need a user in this class because I first check if the user already has a secret, if this is the case, I ask the user if he wants to reset his secret and QR Code. When the secret is decided I ask the user for the code and accept the connection if the code is right. The next time he tries to connect he just needs to check his authentication app then writing the code showed or he can also reset his secret by rescanning a QR Code.

```
Do you wish to create a user (Y or N) ? Y
Each username must be unique or the last user with identical username will be deleted
Enter user's name: user1
Enter user's password: azerty
Do you still want to create more users (Y or N) ? hg
Invalid input you cannot create more users
Do you want to try connecting with your own users (type 1), using the demonstration users (type 2) or qui the application (type 3) ? 1
Enter the username :  user
Enter the password :  azerty
Username: user
Password: ******
Wrong Credentials. Retry in a short time
Login failed
Do you want to try connecting with your own users (type 1), using the demonstration users (type 2) or qui the application (type 3) ? 1
Enter the username :  user1
Enter the password :  azerty
Username: user1
Password: ******
Which method do you prefer to verify that it's you :
- Receiving a phone call (type 1)
- Send a code by email (type 2)
- Send a code by text message (type 3)
- Scan a QRCode in an Authentificator app (type 4)
- Check the code in your Authentificator app (type 5)
1
Please enter your phone number to receive a call:
any number
call to any number
Login successful
Do you want to try connecting with your own users (type 1), using the demonstration users (type 2) or qui the application (type 3) ? fdgjl
Invalid input the application will end
```

In the previous screenshot we can see the creation of a user, the default case of the creation of another user, a wrong username and a successful login with a phone call MFA.

```
Do you wish to create a user (Y or N) ? gsfkdh
Next time type something valid
Do you want to try connecting with your own users (type 1), using the demonstration users (type 2) or qui the application (type 3) ? sfhk
Invalid input the application will end
```

On the previous screenshot we can see the default case of the creation of user and the default case of the login interface.

```
Do you wish to create a user (Y or N) ? Y
Each username must be unique or the last user with identical username will be deleted
Enter user's name: user1
Enter user's password: azerty
Do you still want to create more users (Y or N) ? Y
Each username must be unique or the last user with identical username will be deleted
Enter user's name: user2
Enter user's password: qwerty
Do you still want to create more users (Y or N) ? N
Do you want to try connecting with your own users (type 1), using the demonstration users (type 2) or qui the application (type 3) ? 1
Enter the username :  user1
Enter the password :  azerty
Username: user1
Password: ******
Which method do you prefer to verify that it's you :
- Receiving a phone call (type 1)
- Send a code by email (type 2)
- Send a code by text message (type 3)
- Scan a QRCode in an Authentificator app (type 4)
- Check the code in your Authentificator app (type 5)
2
Please enter your email to receive a code by email:
any email
sent a code to any email
Login successful
Do you want to try connecting with your own users (type 1), using the demonstration users (type 2) or qui the application (type 3) ? dfs
Invalid input the application will end
```

On the previous screenshot we can see the creation of two users with different username and the case “N” to stop creating more users.

```
Do you wish to create a user (Y or N) ? N
Do you want to try connecting with your own users (type 1), using the demonstration users (type 2) or qui the application (type 3) ? 2
Try typing 5
Username: test
Password: ******
Which method do you prefer to verify that it's you :
- Receiving a phone call (type 1)
- Send a code by email (type 2)
- Send a code by text message (type 3)
- Scan a QRCode in an Authentificator app (type 4)
- Check the code in your Authentificator app (type 5)
5
QR Code generated
Please enter the secret code on your authentification app
122222
Two Step Verification failed
There are 2 attempts left

Username: test
Password: ******
Which method do you prefer to verify that it's you :
- Receiving a phone call (type 1)
- Send a code by email (type 2)
- Send a code by text message (type 3)
- Scan a QRCode in an Authentificator app (type 4)
- Check the code in your Authentificator app (type 5)
5
Do you want to use the saved secret (Y or N) ?
Y
Please enter the secret code on your authentification app
224721
Login successful
```

```
Try typing 5 again
Username: test
Password: ******
Which method do you prefer to verify that it's you :
- Receiving a phone call (type 1)
- Send a code by email (type 2)
- Send a code by text message (type 3)
- Scan a QRCode in an Authentificator app (type 4)
- Check the code in your Authentificator app (type 5)
5
Do you want to use the saved secret (Y or N) ?
Y
Please enter the secret code on your authentification app
940180
Login successful
It will not work
Username: test
Password: ******
Wrong Credentials. Retry in a short time
Login failed
Try typing 4
Username: I lost
Password: ********
Which method do you prefer to verify that it's you :
- Receiving a phone call (type 1)
- Send a code by email (type 2)
- Send a code by text message (type 3)
- Scan a QRCode in an Authentificator app (type 4)
- Check the code in your Authentificator app (type 5)
4
QR Code generated
Use the QRCode in your authentificator app and type the code you obtain
529824
Login successful
```

On the last three previous screenshot we can see the demonstration users with a failed connection attempt using a QR Code and an example of generated QR Code

```
Do you wish to create a user (Y or N) ? Y
Each username must be unique or the last user with identical username will be deleted
Enter user's name: user1
Enter user's password: azerty
Do you still want to create more users (Y or N) ? Y
Each username must be unique or the last user with identical username will be deleted
Enter user's name: user1
Enter user's password: qwerty
Do you still want to create more users (Y or N) ? fds
Invalid input you cannot create more users
Do you want to try connecting with your own users (type 1), using the demonstration users (type 2) or qui the application (type 3) ? 1
Enter the username :  user1
Enter the password :  azerty
Username: user1
Password: ******
Wrong Credentials. Retry in a short time
Login failed
Do you want to try connecting with your own users (type 1), using the demonstration users (type 2) or qui the application (type 3) ? 1
Enter the username :  user1
Enter the password :  qwerty
Username: user1
Password: ******
Which method do you prefer to verify that it's you :
- Receiving a phone call (type 1)
- Send a code by email (type 2)
- Send a code by text message (type 3)
- Scan a QRCode in an Authentificator app (type 4)
- Check the code in your Authentificator app (type 5)
3
Please enter your phone number to receive a code by SMS:
any number
send a code to any number
Login successful
Do you want to try connecting with your own users (type 1), using the demonstration users (type 2) or qui the application (type 3) ? 3
```

On the previous screenshot we can see the creation of two users with the same username and when trying to log in with the credentials of the first user the credentials are not correct, but when using the credentials of the second user the credentials are correct. We can also see an example of the third MFA method.

```
Do you wish to create a user (Y or N) ? Y
Each username must be unique or the last user with identical username will be deleted
Enter user's name: user1
Enter user's password: azerty
Do you still want to create more users (Y or N) ? N
Do you want to try connecting with your own users (type 1), using the demonstration users (type 2) or qui the application (type 3) ? 1
Enter the username :  user1
Enter the password :  azerty
Username: user1
Password: ******
Which method do you prefer to verify that it's you :
- Receiving a phone call (type 1)
- Send a code by email (type 2)
- Send a code by text message (type 3)
- Scan a QRCode in an Authentificator app (type 4)
- Check the code in your Authentificator app (type 5)
ghk
Please enter a number between 1 and 5
There are 2 attempts left

Username: user1
Password: ******
Which method do you prefer to verify that it's you :
- Receiving a phone call (type 1)
- Send a code by email (type 2)
- Send a code by text message (type 3)
- Scan a QRCode in an Authentificator app (type 4)
- Check the code in your Authentificator app (type 5)
9
Please enter a valid choice
There are 1 attempts left
```

```
Username: user1
Password: ******
Which method do you prefer to verify that it's you :
- Receiving a phone call (type 1)
- Send a code by email (type 2)
- Send a code by text message (type 3)
- Scan a QRCode in an Authentificator app (type 4)
- Check the code in your Authentificator app (type 5)
9
Login failed
Do you want to try connecting with your own users (type 1), using the demonstration users (type 2) or qui the application (type 3) ? 3
```

On the last two screenshots we can see the NumberFormatException of the login method and the default case of this method. We can also see a case where the MFA attempt fail.

3) General Questions

a. Question 1:

The pattern I find the most useful is the builder because I prefer to avoid having multiple constructors in a single class. The builder pattern is useful because with it one can build an object by choosing which variables he wants to define without having to create more constructors or defining all the unused variables with default value.

This pattern could be useful to perform test because usually when testing complex method one needs to override other methods. And in this case the class may require variables to store the overridden method. If there are only 2 or 3 methods involved in the class a builder isn't useful but if there are more than that, the builder can be useful.

In this design a builder pattern could have been used for the FixedSecretAuthentificator class to reduce the complexity of the constructor because the current implementation contains a constructor with seven arguments that must be repeated in other constructors.

b. Question 2:

The Singleton pattern is a creational pattern used to ensure that a class has only one global instance throughout the program. When a singleton is requested, it checks whether an instance already exists and, in that case, the existing instance will be returned [5].

First the singleton can violate the SRP because for example if the SaveUser class is responsible for saving users and a singleton instance is used of the class SendEmail, the SaveUser class will have two responsibilities which breaks the SRP [7].

Then a singleton introduces a global instance in the project so, when testing, modifying of the singleton may affect the entire program. This pattern is" generally this is used to simplify code, without thinking of the difficulties that will be introduced" [6]. Also, if the program uses several threads, race condition may occur.

4) Unit testing

The following screenshots show the coverage report for the whole project.

26001721 > Bradford.CWW — Source Files — Sessions

Bradford.CWW

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
App		0 %		n/a	2	2	4	4	2	2	1	1
Login		100 %		100 %	0	23	0	73	0	8	0	1
UserInterface		100 %		94 %	1	14	0	60	0	4	0	1
MFALogin		100 %		100 %	0	3	0	7	0	2	0	1
Total	10 of 484	97 %	1 of 45	97 %	3	42	4	144	2	16	1	4

Created with JaCoCo 0.8.13.202504020838

26001721 > Bradford.CWW.MFA — Source Files — Sessions

Bradford.CWW.MFA

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
GUIQrCode		0 %		0 %	7	7	30	30	4	4	1	1
AuthentificatorApp		85 %		60 %	5	16	7	50	2	11	0	1
FixedSecretAuthentificator		100 %		100 %	0	8	0	39	0	4	0	1
RandomSecretAuthentificatorApp		100 %		n/a	0	4	0	21	0	4	0	1
CodeSentByEmail		100 %		n/a	0	2	0	7	0	2	0	1
PhoneCall		100 %		n/a	0	2	0	7	0	2	0	1
CodeSentBySMS		100 %		n/a	0	2	0	7	0	2	0	1
Total	131 of 553	76 %	10 of 23	56 %	12	41	37	161	6	29	1	7

Created with JaCoCo 0.8.13.202504020838

26001721 > Bradford.CWW.assets > User — Sessions

User

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods
User(String, String, String)		100 %		n/a	0	1	0	5	0	1
User()		100 %		n/a	0	1	0	2	0	1
User(String, String)		100 %		n/a	0	1	0	2	0	1
setSecret(String)		100 %		n/a	0	1	0	2	0	1
setPassword(String)		100 %		n/a	0	1	0	2	0	1
setUsername(String)		100 %		n/a	0	1	0	2	0	1
getSecret()		100 %		n/a	0	1	0	1	0	1
getPassword()		100 %		n/a	0	1	0	1	0	1
getUsername()		100 %		n/a	0	1	0	1	0	1
Total	0 of 45	100 %	0 of 0	n/a	0	9	0	18	0	9

Created with JaCoCo 0.8.13.202504020838

Here is the GUIQrCode class which is not tested because this class creates a GUI and GUI testing are neither mandatory nor taught.

```java
public class GUIQrCode {
    private final JFrame frame;
    private final JLabel qrCodeLabel;

    public GUIQrCode(String filepath) {

        frame = new JFrame("QRCode");
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setLayout(new BorderLayout());
        frame.setTitle("QR Code");

        qrCodeLabel = new JLabel("Loading QRCode");
        frame.setSize(400, 400);
        frame.add(qrCodeLabel, BorderLayout.CENTER);
        frame.setLocationRelativeTo(null);
        loadQrCode(filepath);


    }

    private void loadQrCode(String filepath) {
        if (filepath == null || filepath.isEmpty()) {
            qrCodeLabel.setText("No QR code filepath provided");
            return;
        }
        File qrCode = new File(filepath);

        if (!qrCode.exists()) {
            qrCodeLabel.setText("No QR code file exits");
            return;
        }

        ImageIcon qrCodeIcon = new ImageIcon(qrCode.getPath());

        qrCodeLabel.setIcon(qrCodeIcon);
        qrCodeLabel.setText(null);
        frame.pack();
        frame.setLocationRelativeTo(null);
        frame.revalidate();
        frame.repaint();
    }


    public void printQrCode() {
        frame.setVisible(true);
    }

    public void end() {
        frame.dispose();
```

Here is the coverage of the AuthentificationApp class as you can see the only untested method are methods related to GUI creation or closing.

```java
    public AuthentificatorApp(TimeProvider timeProvider,CodeGenerator codeGenerator, QrGenerator qrGenerator,SecretGenerator secretGenerator) {
        this.timeProvider = timeProvider;
        this.codeGenerator = codeGenerator;
        this.qrGenerator = qrGenerator;
        this.secretGenerator = secretGenerator;
    }

    public AuthentificatorApp(TimeProvider timeProvider,CodeGenerator codeGenerator) {
        this(timeProvider,codeGenerator,new ZxingPngQrGenerator(),new DefaultSecretGenerator());
    }

    public void generateQrCode(String secret,String label,String path) throws QrGenerationException,IOException {
        Path = path;

        // secret = "BP26TDZUZ5SVPZJRIHCAUVREO5EWMHHV"
        byte [] imageData= generateQRCodeData(secret,label);

        generateImage(imageData,path);
        if (!testMode) {
            SwingUtilities.invokeLater(() -> {
                        this.guiQrCode= new GUIQrCode(Path);
                        this.guiQrCode.printQrCode();
                    }
            );
        }
    }

    private void generateImage(byte[] imageData,String path) throws IOException {
        // Source - https://stackoverflow.com/q/15477152
        // Posted by Arvind Sridharan, modified by community. See post 'Timeline' for change history
        // Retrieved 2026-03-06, License - CC BY-SA 3.0
        if (imageData == null) {
            throw new IOException("Image data is null");
        }
        if (imageData.length == 0) {
            throw new IOException("Image data is empty");
        }
        FileOutputStream fileOutputStream = new FileOutputStream(path);
        fileOutputStream.write(imageData);
        fileOutputStream.close();
        System.out.println("QR Code generated ");
    }

    public boolean verifyCode(String secret, String code) {
        DefaultCodeVerifier verifier = new DefaultCodeVerifier(codeGenerator, timeProvider);

        end();

        verifier.setTimePeriod(60);
        verifier.setAllowedTimePeriodDiscrepancy(1);

        // secret = the shared secret for the user
        // code = the code submitted by the user
        code = code.trim(); //to remove the space
        return verifier.isValidCode(secret, code);
    }

    public void end() {
        if (guiQrCode != null && !testMode){
            SwingUtilities.invokeLater(() -> {
                this.guiQrCode.end();
            });
        }
    }
```

5) Functional testing

The method I choose for the functional testing is the generateImage method of the AuthentificationApp class

```
@Contract("null, _ -> fail")
private void generateImage(byte[] imageData,String path) throws IOException,IllegalArgumentException {
    // Source - https://stackoverflow.com/q/15477152
    // Posted by Arvind Sridharan, modified by community. See post 'Timeline' for change history
    // Retrieved 2026-03-06, License - CC BY-SA 3.0
    if (imageData == null) {
        throw new IOException("Image data is null");
    }
    if (imageData.length == 0) {
        throw new IOException("Image data is empty");
    }
    if (path == null) {
        throw new IllegalArgumentException("Path is null");
    }
    FileOutputStream fileOutputStream = new FileOutputStream(path);
    fileOutputStream.write(imageData);
    fileOutputStream.close();
    System.out.println("QR Code generated ");
}
```

imageData classes		
EC1	imageData == null	IOException
EC2	imageData.length == 0	IOException
EC3	imageData.length > 0	File Generated
path classes		
EC4	path != null	File generated
EC5	path == null	IllegalArgumentException

TestCase	imageData	path	Expected Result
testImageDataNullShouldThrowsException()	EC1	EC4	IOExcepiton
testImageDataEmptyShouldThrowsException()	EC2	EC4	IOException
testGenerateImageShouldReturnTrue()	EC3	EC4	File generated
testNullPathShouldThrowsIllegalArgumentException()	EC3	EC5	IllegalArgumentException

```java
@Test  rasakhy
public void testImageDataNullShouldThrowsException() {
    TimeProvider timeProvider = new SystemTimeProvider();
    CodeGenerator codeGenerator = new DefaultCodeGenerator();

    QrGenerator qrGenerator = (ZxingPngQrGenerator) generate(data) -> { return null; };

    AuthentificatorApp app = new AuthentificatorApp(timeProvider, codeGenerator, qrGenerator, new DefaultSecretGenerator());

    assertThrows(IOException.class, () -> {
        app.generateQrCode( secret: "BP26TDZUZ5SVPZJRIHCAUVREO5EWMHHV", label: "QrCode", path: "");
    });

}
```

```java
@Test  rasakhy
public void testImageDataEmptyShouldThrowsException() {
    TimeProvider timeProvider = new SystemTimeProvider();
    CodeGenerator codeGenerator = new DefaultCodeGenerator();

    QrGenerator qrGenerator = (ZxingPngQrGenerator) generate(data) -> { return new byte[0]; };

    AuthentificatorApp app = new AuthentificatorApp(timeProvider, codeGenerator, qrGenerator, new DefaultSecretGenerator());

    assertThrows(IOException.class, () -> {
        app.generateQrCode( secret: "BP26TDZUZ5SVPZJRIHCAUVREO5EWMHHV", label: "QrCode", path: "");
    });

}
```

```java
@Test  new *
public void testNullPathShouldThrowsIOException() {

    AuthentificatorApp app = new AuthentificatorApp();

    assertThrows(IOException.class, () -> {
        app.generateQrCode( secret: "BP26TDZUZ5SVPZJRIHCAUVREO5EWMHHV", label: "QrCode", path: null);
    });
}
```

```java
@Test  new *
public void testGenerteImageShouldReturnTrue() {

    String secret = "BP26TDZUZ5SVPZJRIHCAUVREO5EWMHHV";

    try {
        TimeProvider timeProvider =  () -> 10000;
        DefaultCodeGenerator codeGenerator = new DefaultCodeGenerator();

        long counter = timeProvider.getTime() / 60; //found in this link https://github.com/samdjstevens/java-totp/blob/master/to
        String code = codeGenerator.generate(secret, counter);
        SecretGenerator secretGenerator = new SecretGenerator() {  new *
            @Contract(pure = true)
            @Override  new *
            public  @NotNull String generate() { return "BP26TDZUZ5SVPZJRIHCAUVREO5EWMHHV"; }
        };

        AuthentificatorApp app = new AuthentificatorApp(timeProvider, codeGenerator,new ZxingPngQrGenerator(),secretGenerator);
        app.setTestMode(true);

        app.generateQrCode(secret, label: "test", path: "/");
        assertTrue(app.verifyCode(secret,code));
    }
    catch (CodeGenerationException |IOException | QrGenerationException | IllegalArgumentException ex) {
        ex.printStackTrace();
    }


}
```

Boundary value for imageData.length()

Boundary name	value	Description
nullBoundary	null	No data
emptyBoundary	O	Empty data
minimumValid	1	Smallest valid data
Valid	>800	Typical QR data

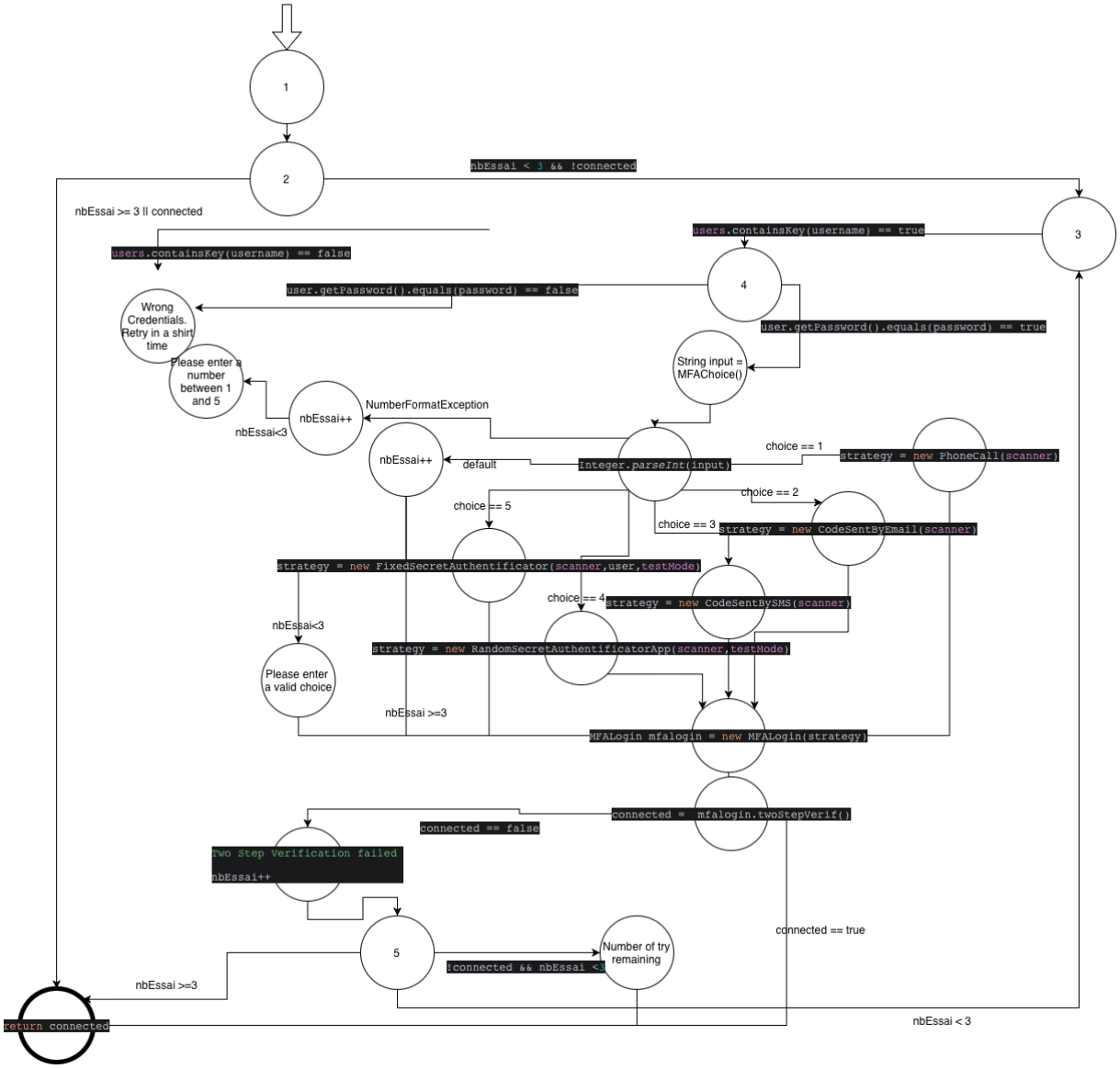
TestCase	imageData value	path	reason
testImageDataNullShouldThrowsException()	null	“/”	Null data
testImageDataEmptyShouldThrowsException()	new byte[0]	“/”	Data empty
testMinimalImageDataShouldReturnTrue()	new byte[1]	“/”	Minimum value
testGenerateImageShouldReturnTrue()	QrData	“/”	Valid input

```java
@Test  new *
public void testMinimalImageDataShouldReturnTrue() {

    String secret = "BP26TDZUZ5SVPZJRIHCAUVREO5EWMHHV";


    try {
        TimeProvider timeProvider =  () -> 10000;
        DefaultCodeGenerator codeGenerator = new DefaultCodeGenerator();

        long counter = timeProvider.getTime() / 60; //found in this link https://github.com/samdjstevens/java-totp/blob/master/totp
        String code = codeGenerator.generate(secret, counter);
        QrGenerator qrGenerator = new ZxingPngQrGenerator() {  new *
            @Contract(value = "_ -> new", pure = true)
            @Override  new *
            public byte @NotNull [] generate(QrData data) {
                return new byte[1];
            }
        };

        AuthentificatorApp app = new AuthentificatorApp(timeProvider, codeGenerator, qrGenerator, new DefaultSecretGenerator());
        app.setTestMode(true);
        app.generateQrCode(secret, label: "test", path: "/");
        app.verifyCode(secret,code);
    }
    catch (CodeGenerationException | QrGenerationException | IOException | IllegalArgumentException ex) {
        ex.printStackTrace();
    }
}
```

6) Structural testing

```java
public boolean login(String username, String password) {  16 usages  rasakhy
    IMFAStrategy strategy = null;
    boolean connected = false;
    int nbEssai = 0;
    while (nbEssai < 3 && !connected) {
        System.out.println(credentialsPrint(username, password));
        if (users.containsKey(username)){
            User user = users.get(username);
            if (user.getPassword().equals(password)) {
                String input = MFAChoice();
                try {
                    int choice = Integer.parseInt(input);
                    switch (choice) {
                        case 1:
                            strategy = new PhoneCall(scanner);
                            break;
                        case 2:
                            strategy = new CodeSentByEmail(scanner);
                            break;
                        case 3:
                            strategy = new CodeSentBySMS(scanner);
                            break;
                        case 4:
                            strategy = new RandomSecretAuthentificatorApp(scanner,testMode);
                            break;
                        case 5:
                            strategy = new FixedSecretAuthentificator(scanner,user,testMode);
                            break;
                        default:
                            nbEssai++;
                            if (nbEssai < 3) {
                                System.out.println("Please enter a valid choice");
                            }
                            break;
                    }
```

```
                    MFALogin mfalogin = new MFALogin(strategy);
                    connected =  mfalogin.twoStepVerif();
                    if (!connected) {
                        System.out.println("Two Step Verification failed ");
                        nbEssai++;
                    }
                }
                catch (NumberFormatException ex) {
                    nbEssai++;
                    if (nbEssai < 3) {
                        System.out.println("Please enter a number between 1 and 5");
                    }
                }
            }
            else {
                System.out.println("Wrong Credentials. Retry in a short time");
                break;
            }
        }
        else {
            System.out.println("Wrong Credentials. Retry in a short time");
            break;
        }
        if (!connected && nbEssai <3) {
            System.out.println("There are "+(3-nbEssai)+" attempts left\n");
        }
    }

    if (connected) {
        System.out.println("Login successful");
    }
    else {
        System.out.println("Login failed");
    }
    return connected;
```



Feasible execution path:

- <u>Username not found:</u> nbEssai <3 && !connected) == true -> users.contains(username) == false -> Wrong Credentials. Retry in a short time -> Login failed -> return false
- <u>Wrong password:</u> (nbEssai<3 && !coonected) == true -> users.contains(username) == true -> user.getPaswword != password -> Wrong Credentials. Retry in a short time -> Login Failed -> return false
- <u>Valid credentials + MFA successful:</u> (nbEssai<3 && !connected) == true -> users.contains(username) == true -> user.getPassword == password -> MFAChoice = “2” -> mfalogin = new MFALogin(strategy)-> mfalogin.TwoStepVerif() == true -> Login Successful -> return true

- Valid credentials + MFA failed: (nbEssai<3 && !connected) == true -> users.contains(username) == true -> user.getPassword == password -> MFAChoice = “2” -> mfalogin = new MFALogin(strategy)-> mfalogin.TwoStepVerif() == false -> nbEssai++ -> MFAChoice = “2” -> mfalogin.TwoStepVerif() == false -> nbEssai++ -> -> MFAChoice = “2” -> mfalogin.TwoStepVerif() == false -> LoginFailed -> return false
- Valid credentials + Invalid MFAChoice: (nbEssai<3 && !connected) == true -> users.contains(username) == true -> user.getPassword == password -> MFAChoice = “9” -> invalid choice ->nbEssai++ -> MFAChoice = “9” -> invalid choice ->nbEssai++ -> MFAChoice = “9” -> nbEssai++ -> login failed -> return false
- Valid credentials + Invalid MFAChoice: (nbEssai<3 && !connected) == true -> users.contains(username) == true -> user.getPassword == password -> MFAChoice = “sgs” -> invalid choice ->nbEssai++ -> MFAChoice = “azv” -> invalid choice ->nbEssai++ -> MFAChoice = “yui” -> nbEssai++ -> login failed -> return false

Statement coverage

Test case	Commentary
TC1 - wrongUsernameShouldReturnFalse()	Cover branch username not found
TC2 - wrongPasswordShouldReturnFalse()	Cover branch wrong password
TC3 - LoginPhoneCallSuccessfulShouldReturnTrue()	Cover login successful
TC4 - LoginFixedAuthentificatorSuccessfulShouldReturnFalse()	Cover login failed and MFA failure
TC5 - testWrongChoiceShouldReturnFalse()	Cover default switch
TC6 - testInvalidChoiceShouldReturnFalse()	Cover catch

```java
@Test  rasakhy
public void wrongPasswordShouldReturnFalse() {
    String username = "";
    String password = "password";

    Login login = new Login( testMode: true);

    assertFalse(login.login(username,password));
}
```

```java
@Test  rasakhy
public void wrongUsernameShouldReturnFalse() {
    String username = "username";
    String password = "";
    Login login = new Login( testMode: true);

    assertFalse(login.login(username,password));
}
```

```java
@Test  rasakhy
public void LoginPhoneCallSuccessfulShouldReturnTrue() {
    String username = "";
    String password = "";

    InputStream input = new ByteArrayInputStream("1\nbhsfkgvbkbk\n".getBytes());

    Login login = new Login(new Scanner(input), testMode: true);

    boolean result = login.login(username,password);
    assertTrue(result);
}
```

```java
@Test  rasakhy
public void LoginFixedAuthentificatorSuccessfulShouldReturnFalse() {
    String username = "";
    String password = "";

    InputStream input = new ByteArrayInputStream("5\nbhsfkgvbkbk\n5\nguodj\n5\nguofsvb\n".getBytes());

    Login login = new Login(new Scanner(input), testMode: true);

    boolean result = login.login(username,password);
    assertFalse(result);
}
```

```java
@Test  rasakhy
public void testWrongChoiceShouldReturnFalse() {
    String username = "";
    String password = "";

    InputStream input = new ByteArrayInputStream("6\n7\n8\n".getBytes());

    Login login = new Login(new Scanner(input), testMode: true);

    boolean result = login.login(username,password);
    assertFalse(result);
}
```

```java
@Test  rasakhy
public void testInvalidChoiceShouldReturnFalse() {
    String username = "";
    String password = "";

    InputStream input = new ByteArrayInputStream("hf\ngfdkh\ngfhskjbfg\n".getBytes());

    Login login = new Login(new Scanner(input), testMode: true);

    boolean result = login.login(username,password);
    assertFalse(result);
}
```

Decision coverage

Decision	True outcome	False outcome
nbEssai<3 && !connected	TC3	TC4, TC5, TC6
users.containsKey(username)	TC3, TC2	TC1
User.getPassword().equals(password)	TC3	TC2
Switch(choice)	TC3	TC4, TC5
connected	TC3	TC4
Integer.parseInt(input)	TC3	TC6
!connected && strategy!=null	TC4	TC5, TC3

Condition coverage

The condition coverage can be applied on the condition: nbEssai<3 && !connected and !connected && strategy!=null

Condition	True	false
nbEssai<3	TC3	TC4
!connected	TC4, TC5	TC3
Strategy !=null	TC3, TC4	TC5

TC3 and TC4 allow the condition nbEssai<3 and !connected to be true and false so, the condition coverage is achieved

TC3 and TC5 allow the condition strategy!=null and !connected to be true and false so, the condition coverage is achieved.

MC/DC coverage:

The MC/DC coverage about this statement: while (nbEssai<3 && !connected)

Name	Test case	nbEssai < 3 (C1)	!connected (C2)	nbEssai<3 && !connnected
T1	TC3	true	true	true
T2	TC3 (after login success)	true	false	false
T3	TC4	false	true	false

We can see that between T1 and T3 the only condition that change is C1 so, C1 can influence the decision.
We can see that between T1 and T2 the only condition that change is C2 so, C2 can influence the decision.
The MC/DC coverage about this statement: while (nbEssai<3 && !connected)

We can see that between T1 and T3 the only condition that change is C1 so, C1 can influence the decision.
We can see that between T1 and T2 the only condition that change is C2 so, C2 can influence the decision.

The MC/DC coverage about this statement: if (strategy!=null && !connected)

Name	Test case	Strategy!=null (C1)	!connected (C2)	Strategy!=null && !connnected
T1	TC4	true	true	true
T2	TC3	true	false	false
T3	TC5	false	true	false
T4		false	false	Infeasible path

We can see that between T1 and T3 the only condition that change is C1 so, C1 can influence the decision.

We can see that between T1 and T2 the only condition that change is C2 so, C2 can influence the decision.

[1] Scanner (java platform SE 8). (2025, October 20).

https://docs.oracle.com/javase/8/docs/api/java/util/Scanner.html

[2] Map (Java platform SE 8). (2025, October 20).

https://docs.oracle.com/javase/8/docs/api/?java/util/Map.html

[3] Time-based One Time Password *(github)*. (2020, November 5).

https://github.com/samdjstevens/java-totp

[4] GeeksforGeeks. (2022, February 15). Introduction to java swing. GeeksforGeeks.

https://www.geeksforgeeks.org/java/introduction-to-java-swing/

[5] Alexander shvets. (2025, January 1). https://refactoring.guru/design-patterns/singleton

[6] user1710031. (n.d.). *Why is Singleton considered an anti-pattern?* Stack Overflow. Retrieved March 12, 2026, from https://stackoverflow.com/questions/12755539/why-is-singleton-considered-an-anti-pattern

[7] Guimarães, D. (2025, March 13). Singleton - Anti pattern? *DEV Community*.

https://dev.to/darlangui/singleton-anti-pattern-18d7